5.1 传输层提供的服务

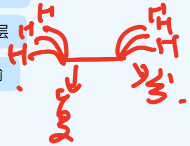
传输层的功能

- 传输层提供进程和进程之间的逻辑通信。
  - 下层的通信过程被隐藏，看起来就像是两个进程直接进行通信 --> 进程之间的逻辑通信

物/链/网—点到点.（设备间）
传及以上 —端到端
进程—进程.

进程 → 端口号 进程 → 端口号.

- 复用和分用
  - 复用
    - 复用：应用层所有的应用进程都可以通过传输层再传输到网络层
    - =发送方不同的进程都可以使用同一个传输层协议进行数据传输
  - 分用
    - 分用：传输层从网络层收到数据后交付指明的应用进程
    - =接收方剥去报文首部后能把数据交付给正确的进程



- 传输层对收到的报文进行差错检测
  - 网络层首部校验和只检验IP数据报首部，需要传输层对数据部分进行差错检测

网络层：只校验首部
传输层：校验首部和数据.

TCP、UDP都可以检测

- 传输层的两种协议
  - TCP（6）
    - 面向连接的传输控制协议
    - 传送数据之前必须建立连接，数据传送结束后要释放连接。不提供广播或多播服务。
    - 缺点 — 由于TCP要提供可靠的面向连接的传输服务，因此不可避免增加了许多开销：确认、流量控制、计时器及连接管理等。
    - 特点 — 可靠，面向连接，时延大，适用于大文件。
  - UDP（17）
    - 无连接的用户数据报协议
    - 传送数据之前不需要建立连接，收到UDP报文后也不需要给出任何确认。
    - 特点 — 不可靠，无连接，时延小，适用于小文件。

对比

传输层的寻址与端口

- 端口号
  - 端口是传输层的SAP，标识主机中的应用进程。 — SAP：服务访问点
  - 端口号长度为16bit,能表示65536个不同的端口号（2^{16}）（16位.）
  - 两大类端口
    - 服务器端使用的端口号
      - 熟知端口号/知名端口号：0-1023 — 给TCP/IP最重要的一些应用程序，让所有用户都知道。
      - 登记端口号：1024-49151 — 为没有熟知端口号的应用程序使用的
    - 客户端使用的端口号
      - 短暂端口号：49152-65535 — 仅在客户进程运行时才动态选择
      - 主机操作系统给正在运行的进程动态分配客户端端口号，进程结束，端口号收回

熟知端口号和登记端口号都是对于Server而言的.

  - 常用端口号

头发头皮.、 虽那么胖

应用程序	FTP	TELNET	SMTP	DNS	TFTP	HTTP	SNMP
熟知端口号	21	23	25	53	69	80	161 斤.
记：	发现 FTP	谈恋爱 TELNET	删好友 SMTP	打电话 DNS		还要再见 HTTP	

（背）

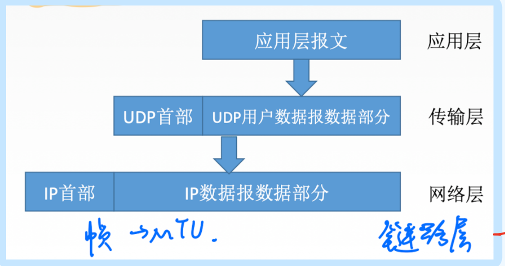
这些都是服务器的端口号，
客户端用的端口是49152~65535端口号.

- 套接字
  - 套接字Socket=（主机IP地址，端口号）（主机 进程.）
  - 在网络中采用发送方和接收方的套接字组合来识别端点，套接字唯一标识了网络中的一个主机和它上面的一个进程。

一只米雨(xhs:463184534)

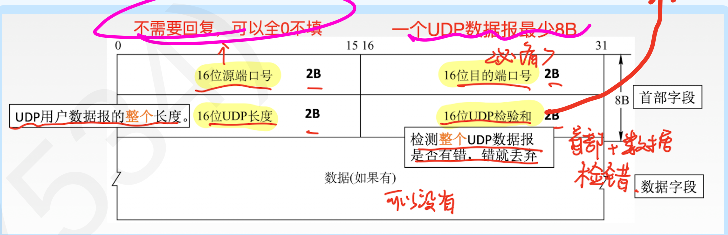
5.2 UDP协议

UDP数据报


应用层报文
应用层
UDP首部
UDP用户数据报数据部分
传输层
IP首部
IP数据报数据部分
网络层
帧 →MTU.
链路层

→以太网 1500B.

- **UDP的主要特点**
  - 1.UDP是无连接的，减少开销和发送数据之前的时延 — 无连接
  - 2.UDP使用最大努力交付，即不保证可靠交付。 — 不可靠
  - 3.UDP是面向报文的，适合一次性传输少量数据的网络应用 — **应用层一次性给UDP多长的报文，UDP就传送多长的报文，既不合并，也不拆分**
  - **4.UDP无拥塞控制，适合很多实时应用**
  - 5.UDP首部开销小
    - **UDP首部8B**
    - TCP首部20B
- UDP首部格式


不需要回复，可以全0不填
一个UDP数据报最少8B
0
15 16
31
必有
16位源端口号
2B
16位目的端口号
2B
8B
首部字段
UDP用户数据报的整个长度。
16位UDP长度
2B
16位UDP检验和
2B
检测整个UDP数据报是否有错，错就丢弃
首部+数据 检错
数据(如果有)
可以没有
数据字段
选用.

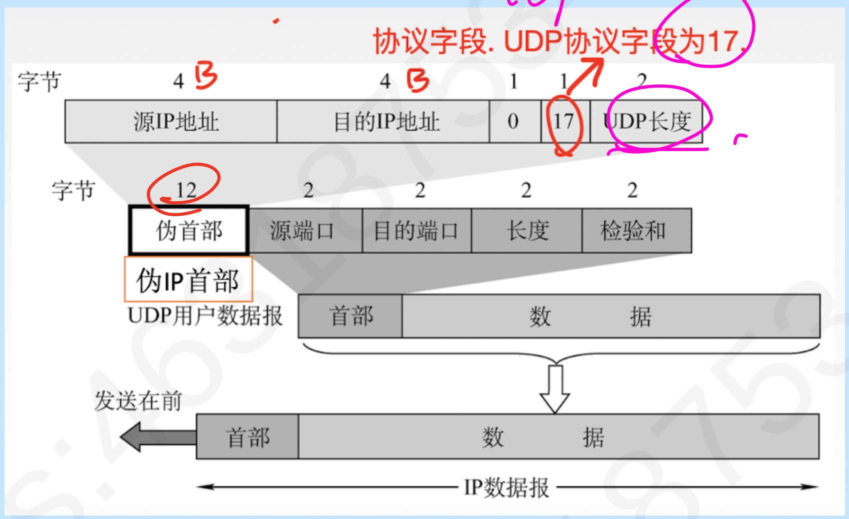
源目端口号，长度检验和

分用时，找不到对应的目的端口号，就丢弃报文，并给发送方发送ICMP"端口不可达 " 差错报告报文。

(终端时参数.

UDP:校验和字段: 检测UDP数据报在传输中是否有错, 有错就丢弃. 该字段是可以选的, 当源主机不想计算校验和时, 则直接令该字段为0

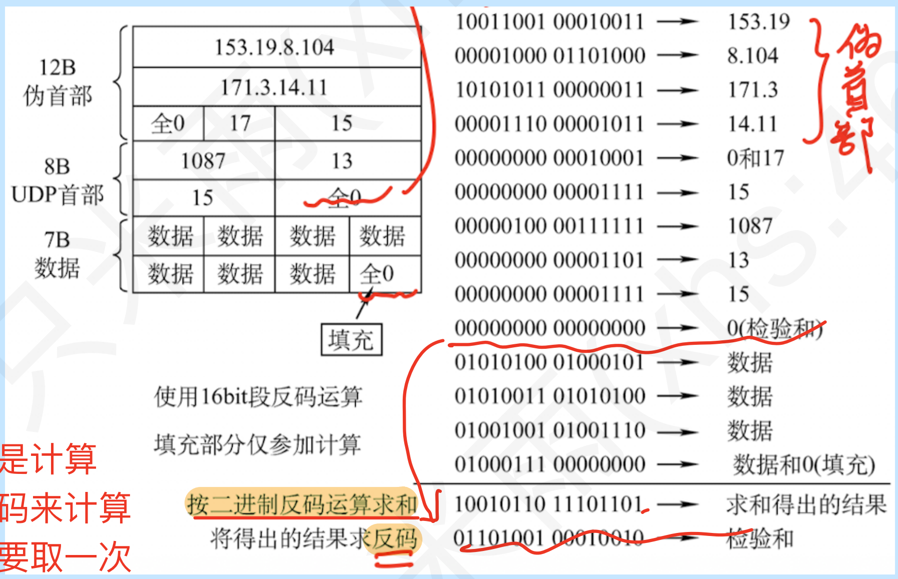
UDP检验


TCP=6.
协议字段. UDP协议字段为17.
字节
4 B
4 B
1
1
2
源IP地址
目的IP地址
0
17
UDP长度
字节
12
2
2
2
2
伪首部
源端口
目的端口
长度
检验和
伪IP首部
UDP用户数据报
首部
数 据
发送在前
首部
数 据
IP数据报

- 伪首部
  - 伪首部包括
    - 源IP地址：32b
    - 目的IP地址：32b
    - 0：1B
    - 17：1B — 封装UDP报文的IP数据报首部协议字段是17。
    - UPD长度：16b — UDP首部8B+数据部分长度（不包括伪首部）。
  - 伪首部只有在计算检验和时才出现， 不向下传送也不向上递交。

12B.

TCP:6.


12B
伪首部
8B
UDP首部
7B
数据
填充
使用16bit段反码运算
填充部分仅参加计算
伪首部

153.19.8.104			
171.3.14.11			
全0	17	15	
1087		13	
15		全0	
数据	数据	数据	数据
数据	数据	数据	全0

10011001 00010011 →	153.19
00001000 01101000 →	8.104
10101011 00000011 →	171.3
00001110 00001011 →	14.11
00000000 00010001 →	0和17
00000000 00001111 →	15
00000100 00111111 →	1087
00000000 00001101 →	13
00000000 00001111 →	15
00000000 00000000 →	0(检验和)
01010100 01000101 →	数据
01010011 01010100 →	数据
01001001 01001110 →	数据
01000111 00000000 →	数据和0(填充)
10010110 11101101 →	求和得出的结果
01101001 00010010 →	检验和

按二进制反码运算求和

将得出的结果求反码

（看看就行）

二进制反码运算就和, 就是计算时, 将数据全都换成其反码来计算结果; 结算出来的结果还要取一次反码, 才是检验和数据

- 在发送端
  - 1.填上伪首部
  - 2.全0填充检验和字段
  - 3.全0填充数据部分(UDP数据报要看成许多4B的字串接起来) 在发送端：
  - 4.伪首部+首部+数据部分采用二进制反码求和
  - 5.把和求反码填入检验和字段
  - 6.去掉伪首部，发送
- 在接收端
  - 1.填上伪首部
  - 2.伪首部+首部+数据部分采用二进制反码求和
  - 3.结果全为1则无差错，否则丢弃数据报/交给应用层附上出差错的警告。 — 全为1则无差错

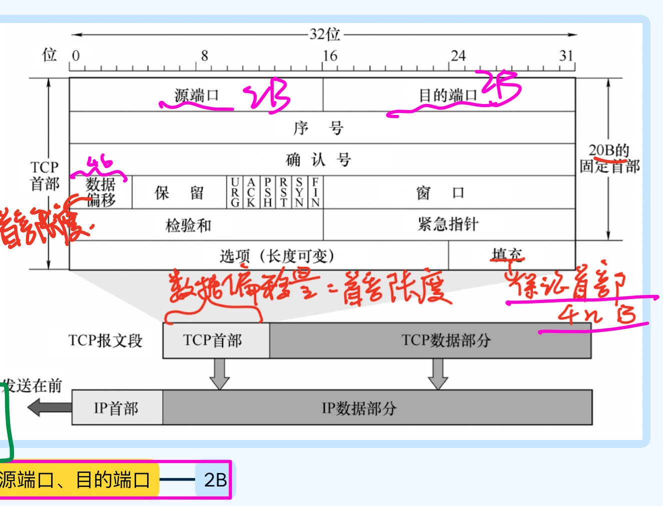
5.3 TCP协议

TCP协议的特点

- 1.TCP是面向连接（虚连接）的传输层协议。
- 2.每一条TCP连接只能有两个端点，每一条TCP连接只能是点对点的。
  - 不能广播，多播
- 3.TCP提供可靠交付的服务，无差错、不丢失、不重复、按序到达。
  - 可靠有序，不丢不重
- 4.TCP 提供全双工通信。
  - 发送缓存
    - 准备发送的数据&已发送但尚未收到确认的数据 → swnd
  - 接收缓存
    - 按序到达但尚未被接受应用程序读取的数据&不按序到达的数据 → rwnd.
- 5.TCP面向字节流
  - TCP 把应用程序交下来的数据看成仅仅是一连串的无结构的字节流
    - 流：流入到进程或从进程流出的字节序列

TCP报文段

4B整数倍
↑
TCP、IP首部最小长度20B，最大长度60B.
IPv6首部固定40B
↓
8B整数倍.


32位
位 0 8 16 24 31
源端口 2B
目的端口 2B
序 号
确 认 号
TCP首部
数据偏移
保 留
URG ACK PSH RST SYN FIN
窗 口
检验和
紧急指针
选项（长度可变）
填充
20B的固定首部
首部长度.
数据偏移量=首部长度
保证首部4n B
TCP报文段
TCP首部
TCP数据部分
发送在前
IP首部
IP数据部分

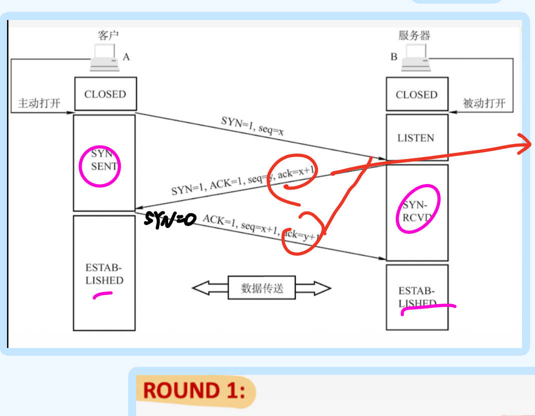
UDP:8B.

- TCP报文段的首部格式
  - 源端口、目的端口 —— 2B
  - 序号 —— 在一个TCP连接中传送的字节流中的每一个字节都按顺序编号，本字段表示本报文段所发送数据的第一个字节的序号。
  - 确认号 —— 期望收到对方下一个报文段的第一个数据字节的序号。若确认号为N,则证明到序号N-1为止的所有数据都已正到。
    - ACK=1
  - 数据偏移（首部长度）
    - TCP报文段的数 据起始处距离TCP报文段的起始处有多远，以4B位单位，即1个数值是4B。
      - 首部为4B的整数倍.
    - 4位
      - 单位4B. ⇒首部最长4×15=60B
  - 首4，
  - 紧急位URG (urgent) —— URG=1表明紧急指针有效，告诉系统报文段中有紧急数据，应尽快传输
  - 确认位ACK (acknowledgement) —— ACK=1表明确认号字段有效，ACK=0表明无效
  - 推送位PSH (push) —— 接收TCP收到PSH=1的报文段，就尽快交付给进程，不等整个缓存存满再传
  - 复位位RST (reset) —— RST=1
    - 表明TCP连接中出现 严重差错，必须释放连接，然后再重新 建立传输链接
    - 也用于拒绝非法报文和非法连接
  - 同步位SYN (synchronization) —— SYN=1时，表明是一个连接 请求/连接接受报文
  - 终止位FIN (finish) —— FIN=1表明此报文段的发送方的数据发送完毕，要求释放连接
  - 建立连接后所有报文ACK都置为1
  - 窗口字段 —— 指的是发送本报文段的一方的接收窗口，即现在允许对方发送的数据量。
    - rwnd.
    - swnd = min{cwnd, rwnd}
  - 检验和：2B
    - 检验首部+数据，检验时要加 上12B伪首部 12B.
    - 伪首部和UDP中的一样，不同的是TCP中伪首部的第四个字段协议字段为6，相对于UDP则为17
  - 紧急指针 —— URG=1时才有意义，指出 本报文段中紧急数据的字节数
  - 选项字段：长度可变
    - 最大报文段长度MSS
    - 窗口扩大、时间戳、选择确认

12B伪首部 TCP、UDP都要用.

TCP连接管理

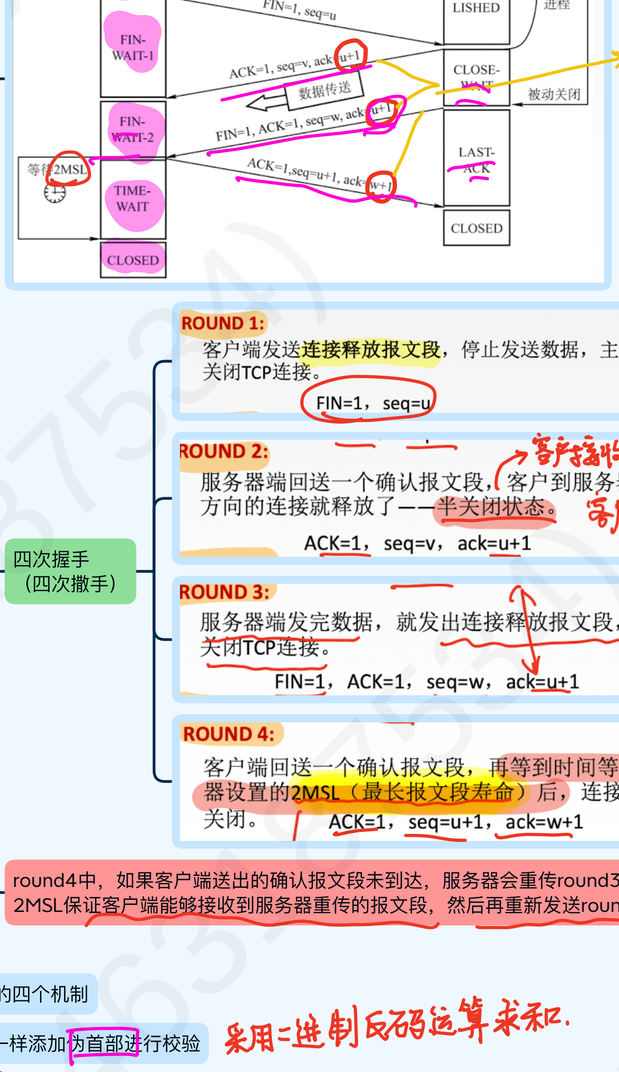
- TCP连接建立
  - TCP连接的建立采用客户服务器方式，主动发起连接建立的应用进程叫做客户，而被动等待连接建立的应用进程叫服务器。
    - client-server
    - 主动发起连接 —— client
    - 被动连接 —— server


客户
A
服务器
B
主动打开
被动打开
CLOSED
CLOSED
LISTEN
SYN-SENT
SYN-RCVD
SYN=1, seq=x
SYN=1, ACK=1, seq=y, ack=x+1
SYN=0
ACK=1, seq=x+1, ack=y+1
ESTAB-LISHED
ESTAB-LISHED
数据传送

说明建立连接的报文段只含一个字节的数据.

SYN=1, seq=x.
Ack=1, ack=x+1
SYN=1 . seq=y
Ack=1, ack=y+1
seq=x+1
应用层数据在第3次握手时可发

  - 三次握手
    - ROUND 1: 客户端发送连接请求报文段，无应用层数据。
      - SYN=1，seq=x(随机)
    - ROUND 2: 服务器端为该TCP连接分配缓存和变量，并向客户端返回确认报文段，允许连接，无应用层数据。
      - SYN=1，ACK=1，seq=y(随机)，ack=x+1 → 搭配使用
    - ROUND 3: 客户端为该TCP连接分配缓存和变量，并向服务器端返回确认的确认，可以携带数据。
      - SYN=0，ACK=1，seq=x+1，ack=y+1
    - 连接后就不用SYN了.
    - 只有在连接请求和连接请求确认报文段中，SYN=1
  - SYN洪泛攻击
    - SYN洪泛攻击发生在OSI第四层，这种方式利用TCP协议的特性，就是三次握手。攻击者发送 TCP SYN，SYN是TCP三次握手中的第一个数据包，而当服务器返回ACK后，该攻击者就不对其进行再确认，那这个TCP连接就处于挂起状态，也就是所谓的半连接状态，服务器收不到再确认的话，还会重复发送ACK给攻击者。这样更加会浪费服务器的资源。攻击者就对服务器发送非常大量的这种TCP连接，由于每一个都没法完成三次握手，所以在服务器上，这些 TCP连接会因为挂起状态而消耗CPU和内存，最后服务器可能死机，就无法为正常用户提供了。
    - 解决办法 —— SYN cookie
- TCP连接释放


客户
A
服务器
B
主动关闭
ESTAB-LISHED
数据传送
ESTAB-LISHED
通知应用进程
FIN=1, seq=u
FIN-WAIT-1
ACK=1, seq=v, ack=u+1
CLOSE-WAIT
数据传送
被动关闭
FIN-WAIT-2
FIN=1, ACK=1, seq=w, ack=u+1
LAST-ACK
等待2MSL
ACK=1,seq=u+1, ack=w+1
TIME-WAIT
CLOSED
CLOSED

说明结束连接的报文段只含有一个字节的数据

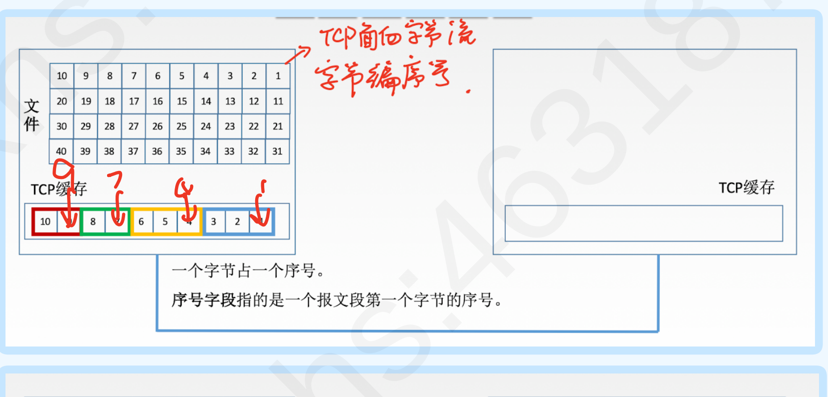
FIN=1, seq=u
Ack=1, ack=u+1
seq=v .
FIN=1, seq=w
Ack=1, ack=u+1
seq=u+1 ,
Ack=1, ack=w+1

  - 四次握手（四次挥手）
    - ROUND 1: 客户端发送连接释放报文段，停止发送数据，主动关闭TCP连接。
      - FIN=1，seq=u
      - 但不回复任何Ack
    - ROUND 2: 服务器端回送一个确认报文段，客户到服务器这个方向的连接就释放了——半关闭状态。
      - 客户端接收服务器的数据
      - 客户 ←/→ 服务器
      - ACK=1，seq=v，ack=u+1
    - ROUND 3: 服务器端发完数据，就发出连接释放报文段，主动关闭TCP连接。
      - FIN=1，ACK=1，seq=w，ack=u+1
    - ROUND 4: 客户端回送一个确认报文段，再等到时间等待计时器设置的2MSL（最长报文段寿命）后，连接彻底关闭。
      - ACK=1，seq=u+1，ack=w+1
  - round4中，如果客户端送出的确认报文段未到达，服务器会重传round3中的报文段，2MSL保证客户端能够接收到服务器重传的报文段，然后再重新发送round4的确认

TCP可靠传输

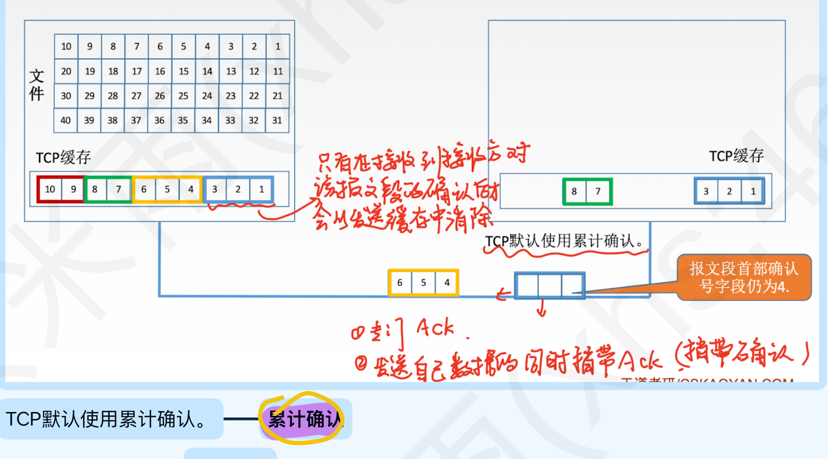
校验 序号
确认 重传.

- TCP保证可靠传输的四个机制
- 校验 —— 与UDP一样添加伪首部进行校验
  - 采用二进制反码运算求和.
- 序号
  - 一个字节占一个序号。 —— 序号字段指的是一个报文段第一个字节的序号。


文件
TCP缓存
TCP缓存
TCP面向字节流
字节编序号.

一个字节占一个序号。

序号字段指的是一个报文段第一个字节的序号。

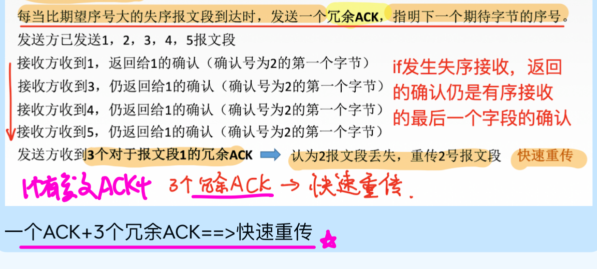

文件
TCP缓存
TCP缓存
报文段首部确认号字段仍为4.

TCP默认使用累计确认。

只有在接收到接收方对该报文段的确认时会从发送缓存中消除

①到Ack.
②发送自己数据的同时捎带Ack（捎带确认）

- 确认 —— TCP默认使用累计确认。 —— 累计确认
- 重传
  - 引起重传的事件
    - 超时重传
    - 冗余ACK
  - 超时重传
    - RTT(Round-trip time) —— 报文段发出的时间与收到确认的时间的差值
    - TCP采用自适应算法，动态改变重传时间RTTs（加权平均往返时间） RTTs.
    - 详细说明
      - 为了计算超时计时器的重传时间，TCP采用自适应算法，他记录一个报文段发出的时间，以及受到相应确认的时间，这两个时间之差称为报文段的往返时间（RTT）。TCP保留了RTT的一个加权平均往返时间RTTs，它会随新测量的RTT样本值的变化而变化
      - 超时计数器设置的超时重传时间（Retransmission Time-Out，RTO）应该略大于RTTs
  - 冗余ACK（冗余确认） 快重传.
    - 每当比期望序号大的失序报文段到达时，发送一个冗余ACK，指明下一个期待字节的序号



冗余ACK（冗余确认）

每当比期望序号大的失序报文段到达时，发送一个冗余ACK，指明下一个期待字节的序号。

发送方已发送1，2，3，4，5报文段

接收方收到1，返回给1的确认（确认号为2的第一个字节）

接收方收到3，仍返回给1的确认（确认号为2的第一个字节）

接收方收到4，仍返回给1的确认（确认号为2的第一个字节）

接收方收到5，仍返回给1的确认（确认号为2的第一个字节）

发送方收到3个对于报文段1的冗余ACK ➡ 认为2报文段丢失，重传2号报文段 快速重传

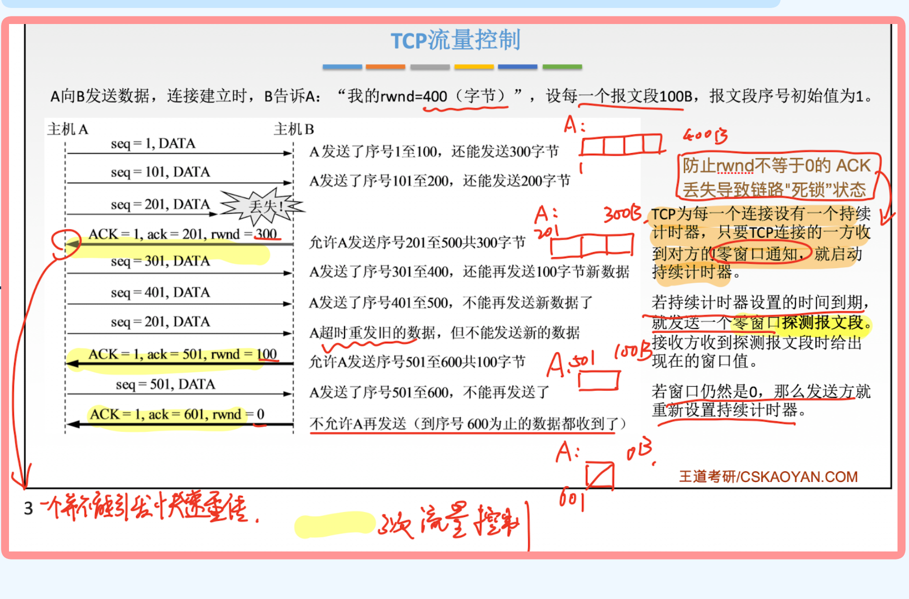
if发生失序接收，返回的确认仍是有序接收的最后一个字段的确认

1个正常ACK+3个冗余ACK → 快速重传.

    - 一个ACK+3个冗余ACK==>快速重传

TCP流量控制

- TCP利用滑动窗口机制实现流量控制
- 在通信过程中，接收方根据自己接收缓存的大小，动态地调整发送方的发送窗口大小，即接收窗口rwnd（接收方设置确认报文段(或数据报文段捎带确认)的窗口字段来将rwnd通知给发送方），发送方的发送窗口取接收窗口 rwnd和拥塞窗口cwnd的最小值。
- 发送窗口大小可以动态调整
  - 接收窗口：rwnd
  - 拥塞窗口：cwnd
  - 发送窗口：swind —— `swind = min{rwnd, cwnd}`
- 窗口字段：16b
  - 窗口字段可以设为0，此时发送方无法将数据发给接收方，接收方可以在该段时间内将接收缓存内的数据交给上层应用层进行数据处理，直到全部/部分数据处理完成，接收方就会返回一个新的报文段（窗口字段不为0）告知发送方可以传输数据以及可接收数据的窗口大小，



TCP流量控制

A向B发送数据，连接建立时，B告诉A：“我的rwnd=400（字节）”，设每一个报文段100B，报文段序号初始值为1。

主机A
主机B
seq = 1, DATA
seq = 101, DATA
seq = 201, DATA
丢失!
ACK = 1, ack = 201, rwnd = 300
seq = 301, DATA
seq = 401, DATA
seq = 201, DATA
ACK = 1, ack = 501, rwnd = 100
seq = 501, DATA
ACK = 1, ack = 601, rwnd = 0
A: 400B
A: 300B 201
A: 501 100B
A: 0B 601
王道考研/CSKAOYAN.COM

A发送了序号1至100，还能发送300字节

A发送了序号101至200，还能发送200字节

允许A发送序号201至500共300字节

A发送了序号301至400，还能再发送100字节新数据

A发送了序号401至500，不能再发送新数据了

A超时重发旧的数据，但不能发送新的数据

允许A发送序号501至600共100字节

A发送了序号501至600，不能再发送了

不允许A再发送（到序号 600为止的数据都收到了）

防止rwnd不等于0的 ACK丢失导致链路"死锁"状态

TCP为每一个连接设有一个持续计时器，只要TCP连接的一方收到对方的零窗口通知，就启动持续计时器。

若持续计时器设置的时间到期，就发送一个零窗口探测报文段。接收方收到探测报文段时给出现在的窗口值。

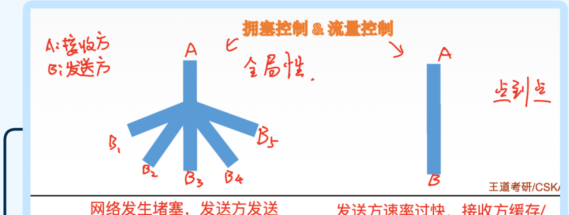
若窗口仍然是0，那么发送方就重新设置持续计时器。

3 一个新确认引发快速重传.

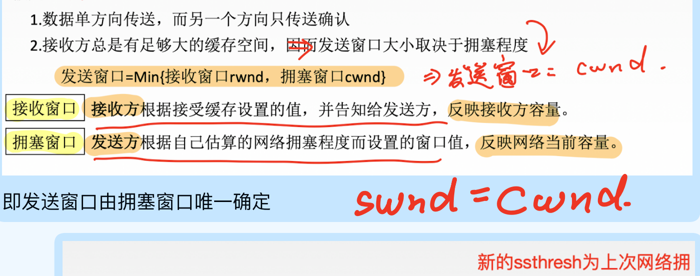
3次流量控制

TCP拥塞控制

- 出现拥塞的条件
  - 对资源需求的总和 >可用资源
  - 网络中有许多资源同时呈现供应不足 ➡ 网络性能变坏 ➡ 网络吞吐量将随输入负荷增大而下降
- 拥塞控制 & 流量控制的区别


拥塞控制 & 流量控制
A:接收方
B:发送方
全局性.
点到点
A
B1
B2
B3
B4
B5
A
B
王道考研/CSK
网络发生堵塞，发送方发送的数据迟迟无法到达接收方
发送方速率过快，接收方缓存/窗口空间不足，来不及接收

  - 拥塞控制
    - 拥塞控制防止过多的数据注入到网络中，使路由器或链路不致过载
    - 拥塞控制是一个全局性的过程，涉及到所有主机、所有路由器，以及降低网络传输性能的所有因素
  - 流量控制
    - 流量控制往往指点对点通信量的控制，是个端对端的问题
    - 流量控制就是抑制发送端发送数据的速率，以便使接收端来的及接收
- 发送窗口swnd=Min{接收窗口rwnd,拥塞窗口cwnd} —— `swind = min{rwnd, cwnd}`
- 拥塞控制过程
  - 一个传输轮次
    - 发送了一批报文段并收到它们的确认的时间。
    - 一个往返时延RTT
    - 开始发送一批拥塞窗口内的报文段到开始发送下一批拥塞窗口内的报文段时间。
    - 三种定义等价
  - 假定



假定:<方便讨论>

1.数据单方向传送，而另一个方向只传送确认

2.接收方总是有足够大的缓存空间，因而发送窗口大小取决于拥塞程度

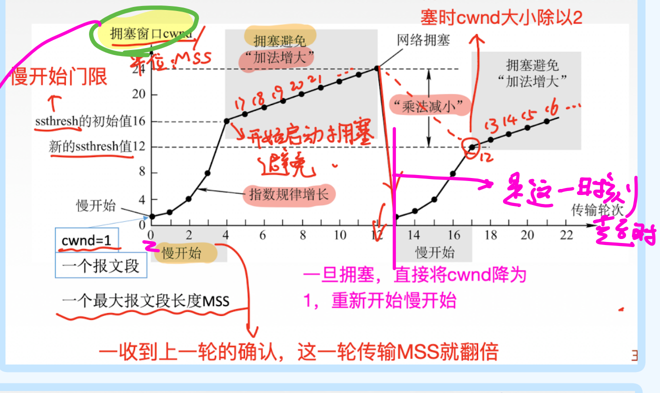
发送窗口=Min{接收窗口rwnd，拥塞窗口cwnd} ⇒发送窗口= cwnd.

接收窗口	**接收方**根据接受缓存设置的值，并告知给发送方，反映接收方容量。
拥塞窗口	**发送方**根据自己估算的网络拥塞程度而设置的窗口值，反映网络当前容量。

    - 即发送窗口由拥塞窗口唯一确定 swnd=cwnd.

swnd=min{cwnd, rwnd}

  - 慢开始 & 拥塞避免



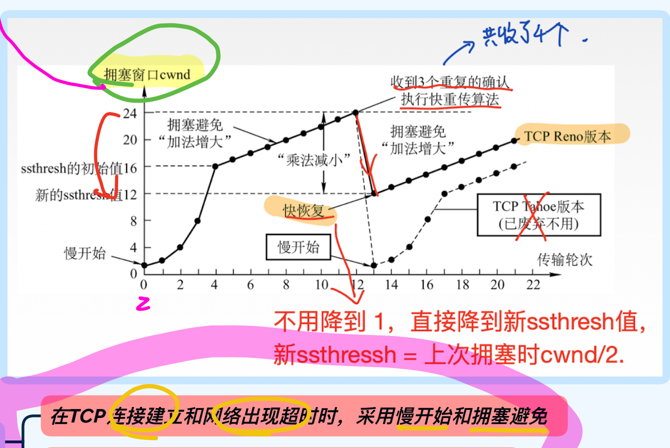
这里的纵坐标是指 cwnd

ssthresh.

这时 慢开始+拥塞避免

超时时将 cwnd 设为1 是不需消耗传输轮次RTT的.

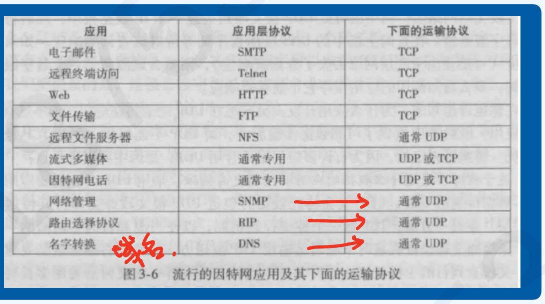
  - 快重传 & 快恢复



快重传+快恢复
↓
1ACK+3个冗余ACK.

  - 四种方法总结
    - *在TCP连接建立和网络出现超时时，采用慢开始和拥塞避免*
    - *当发送方接收到冗余ACK时，采用快重传和快恢复算法*

《自顶向下》补充



应用	应用层协议	下面的运输协议
电子邮件	SMTP	TCP
远程终端访问	Telnet	TCP
Web	HTTP	TCP
文件传输	FTP	TCP
远程文件服务器	NFS	通常 UDP
流式多媒体	通常专用	UDP 或 TCP
因特网电话	通常专用	UDP 或 TCP
网络管理	SNMP	通常 UDP
路由选择协议	RIP	通常 UDP
名字转换 域名.	DNS	通常 UDP

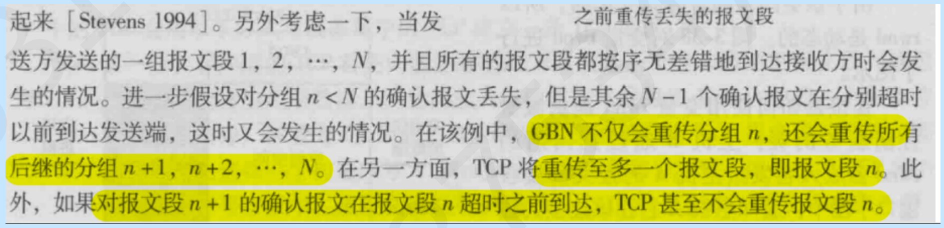
图 3-6　流行的因特网应用及其下面的运输协议

最大报文段长度MSS

- TCP报文段
- 最大报文段长度MSS 不包含TCP首部
- 若使用以太网，一般=1500-40B=1460B

40B = IP头 + TCP头.

GBN与TCP的区别



起来［Stevens 1994］。另外考虑一下，当发　　　　之前重传丢失的报文段

送方发送的一组报文段 1，2，…，N，并且所有的报文段都按序无差错地到达接收方时会发生的情况。进一步假设对分组 $n<N$ 的确认报文丢失，但是其余 $N-1$ 个确认报文在分别超时以前到达发送端，这时又会发生的情况。在该例中，GBN 不仅会重传分组 n，还会重传所有后继的分组 $n+1$，$n+2$，…，N。在另一方面，TCP 将重传至多一个报文段，即报文段 n。此外，如果对报文段 $n+1$ 的确认报文在报文段 n 超时之前到达，TCP 甚至不会重传报文段 n。

TCP与GBN都是累积确认

但：GBN重传未收到那一帧 及之后的所有发过的帧

TCP只重传丢的那一个报文段

客户TCP经历的典型TCP状态序列

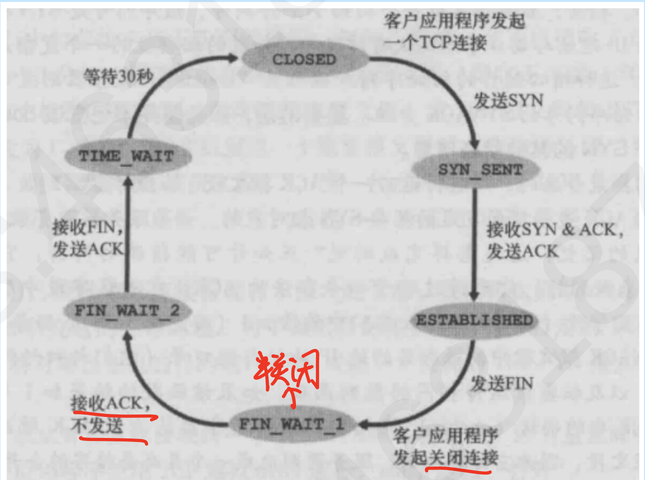
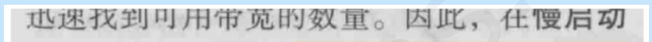

客户应用程序发起
一个TCP连接
CLOSED
发送SYN
SYN_SENT
接收SYN & ACK，
发送ACK
ESTABLISHED
发送FIN
关闭
FIN_WAIT_1
客户应用程序
发起关闭连接
接收ACK，
不发送
FIN_WAIT_2
接收FIN，
发送ACK
TIME_WAIT
等待30秒

图 3-41　客户 TCP 经历的典型的 TCP 状态序列

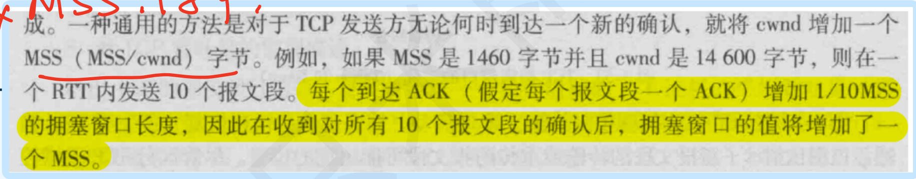
慢开始具体机制



迅速找到可用带宽的数量。因此，在慢启动（slow-start）状态，cwnd 的值以 1 个 MSS 开始并且每当传输的报文段首次被确认就增加 1 个 MSS。在图 3-51 所示的例子中，TCP 向网络发送第一个报文段并等待一个确认。当该确认到达时，TCP 发送方将拥塞窗口增加一个 MSS，并发送出两个最大长度的报文段。这两个报文段被确认，则发送方对每个确认报文段将拥塞窗口增加一个 MSS，使得

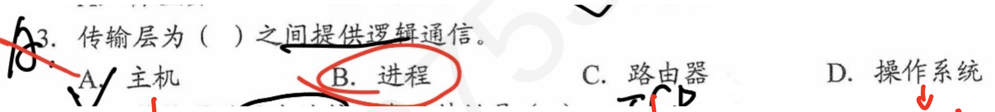
- 1 --> 1+1 --> 1+1+1+1 -->...
- 1 --> 2 --> 4 -->...

拥塞避免具体机制



$\frac{MSS}{cwnd} \times MSS$ 个字节,

成。一种通用的方法是对于 TCP 发送方无论何时到达一个新的确认，就将 cwnd 增加一个 MSS（MSS/cwnd）字节。例如，如果 MSS 是 1460 字节并且 cwnd 是 14 600 字节，则在一个 RTT 内发送 10 个报文段。每个到达 ACK（假定每个报文段一个 ACK）增加 1/10MSS 的拥塞窗口长度，因此在收到对所有 10 个报文段的确认后，拥塞窗口的值将增加了一个 MSS。

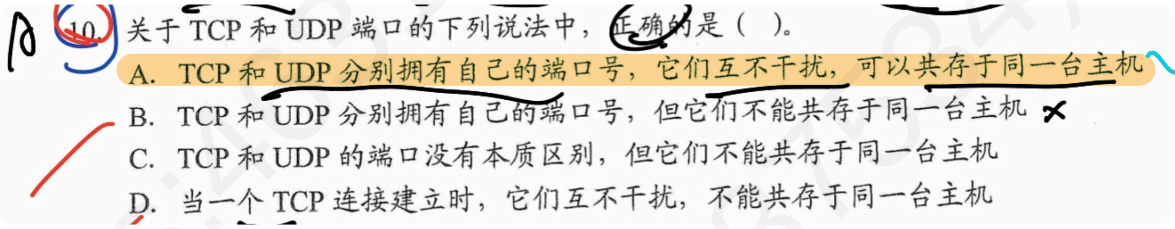


B 3. 传输层为（ ）之间提供逻辑通信。

A. 主机　　B. 进程　　C. 路由器　　D. 操作系统

A → 网络层　　C → 网络层　　D → 涉及多个层次.

传输层为进程之间提供端到端的通信.
物/链/网 → 点到点... 设备间通信.



D 10. 关于 TCP 和 UDP 端口的下列说法中，正确的是（ ）。

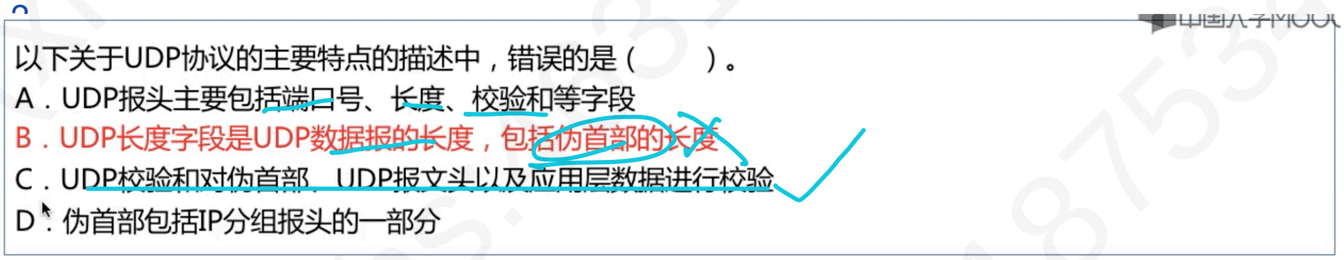
A. TCP 和 UDP 分别拥有自己的端口号，它们互不干扰，可以共存于同一台主机 ✓

B. TCP 和 UDP 分别拥有自己的端口号，但它们不能共存于同一台主机 ×

C. TCP 和 UDP 的端口没有本质区别，但它们不能共存于同一台主机

D. 当一个 TCP 连接建立时，它们互不干扰，不能共存于同一台主机

5.2



以下关于UDP协议的主要特点的描述中，错误的是（ ）。

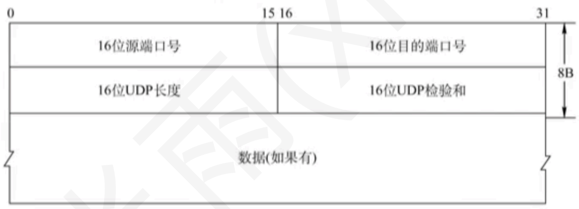
A．UDP报头主要包括端口号、长度、校验和等字段

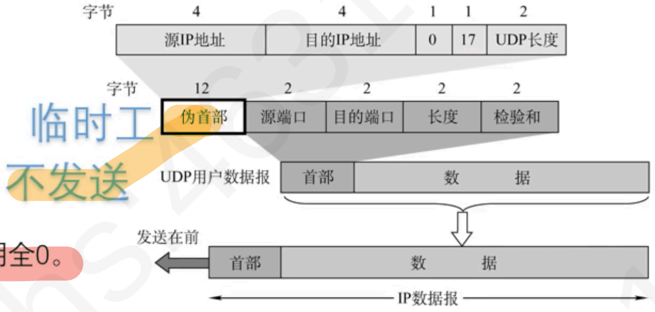
B．UDP长度字段是UDP数据报的长度，包括伪首部的长度 ×

C．UDP校验和对伪首部、UDP报文头以及应用层数据进行校验 ✓

D．伪首部包括IP分组报头的一部分

UDP数据报格式


0
15 16
31
16位源端口号
16位目的端口号
16位UDP长度
16位UDP检验和
8B
数据(如果有)


字节
4
4
1
1
2
源IP地址
目的IP地址
0
17
UDP长度
字节
12
2
2
2
2
伪首部
源端口
目的端口
长度
检验和
UDP用户数据报
首部
数　据
发送在前
首部
数　据
IP数据报

临时工

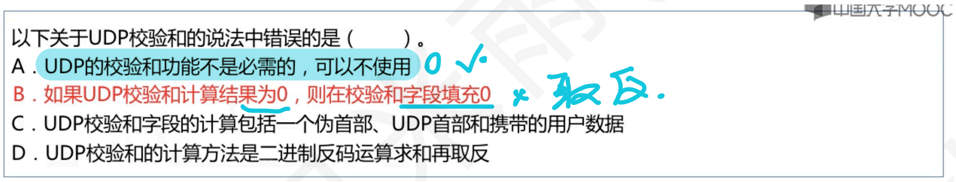
不发送

1）源端口号：在需要对方回信时选用。不需要时可用全0。

2）目的端口 号：这在终点交付报文时必须要使用到。

3）长度 ：UDP数据报的长度（包括首部和数据），其最小值是8（仅有首部）。

4）校验和 ：检测UDP数据报在传输中是否有错，有错就丢弃。
【可选】 当源主机不想计算校验和，则令该字段为0.



以下关于UDP校验和的说法中错误的是（ ）。

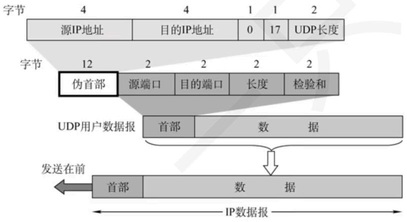
A．UDP的校验和功能不是必需的，可以不使用 0 ✓

B．如果UDP校验和计算结果为0，则在校验和字段填充0 × 取反.

C．UDP校验和字段的计算包括一个伪首部、UDP首部和携带的用户数据

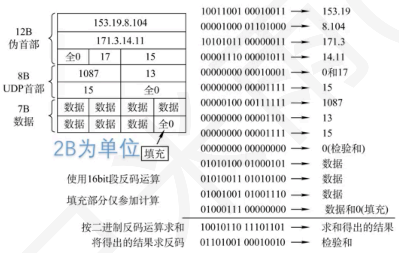
D．UDP校验和的计算方法是二进制反码运算求和再取反

8. B 计算结果为0还要取一次反才是校验和字段的填充


字节
4
4
1
1
2
源IP地址
目的IP地址
0
17
UDP长度
字节
12
2
2
2
2
伪首部
源端口
目的端口
长度
检验和
UDP用户数据报
首部
数　据
发送在前
首部
数　据
IP数据报

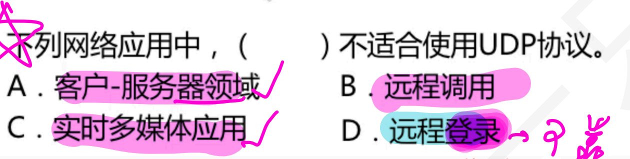
校验和 ：检测UDP数据报在传输中是否有错，有错就丢弃。
【可选】 当源主机不想计算校验和，则令该字段为0。

计算方法：
二进制反码运算求和再取反。


12B
伪首部
153.19.8.104
171.3.14.11
全0
17
15
8B
UDP首部
1087
13
15
全0
7B
数据
数据
数据
数据
数据
数据
数据
数据
全0
填充
使用16bit段反码运算
填充部分仅参加计算
按二进制反码运算求和
将得出的结果求反码

2B为单位

```
10011001 00010011 → 153.19
00001000 01101000 → 8.104
10101011 00000011 → 171.3
00001110 00001011 → 14.11
00000000 00010001 → 0和17
00000000 00001111 → 15
00000100 00111111 → 1087
00000000 00001101 → 13
00000000 00001111 → 15
00000000 00000000 → 0(检验和)
01010100 01000101 → 数据
01010011 01010100 → 数据
01001001 01001110 → 数据
01000111 00000000 → 数据和0(填充)
10010110 11101101 → 求和得出的结果
01101001 00010010 → 检验和
```



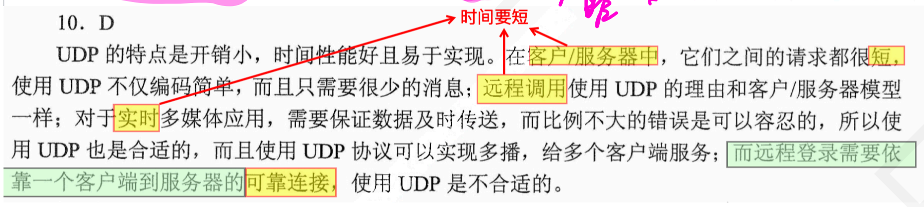
下列网络应用中，（ ）不适合使用UDP协议。

A．客户-服务器领域 ✓　　B．远程调用

C．实时多媒体应用 ✓　　D．远程登录 → 可靠.

TCP	有三次握手，有确认，有流量控制	面向连接、可靠	慢	HTTP
UDP	无确认，无流量控制	无连接、不可靠	快	QQ

开销小，时间性能好，易于实现



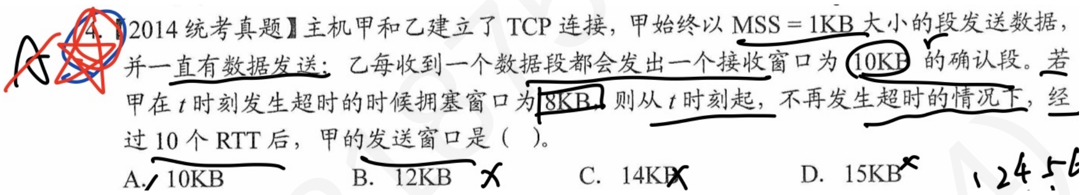
时间要短

10．D

UDP 的特点是开销小，时间性能好且易于实现。在客户/服务器中，它们之间的请求都很短，使用 UDP 不仅编码简单，而且只需要很少的消息；远程调用使用 UDP 的理由和客户/服务器模型一样；对于实时多媒体应用，需要保证数据及时传送，而比例不大的错误是可以容忍的，所以使用 UDP 也是合适的，而且使用 UDP 协议可以实现多播，给多个客户端服务；而远程登录需要依靠一个客户端到服务器的可靠连接，使用 UDP 是不合适的。

10.要求时间短, 实时-->UDP

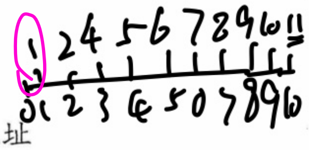
5.3



A 4.【2014 统考真题】主机甲和乙建立了 TCP 连接，甲始终以 MSS = 1KB 大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10KB 的确认段。若甲在 t 时刻发生超时的时候拥塞窗口为 8KB，则从 t 时刻起，不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是（　）。

A. 10KB　　B. 12KB　　C. 14KB　　D. 15KB

1 2 4 5 6



，地址

就是你认为的从那个时候算起

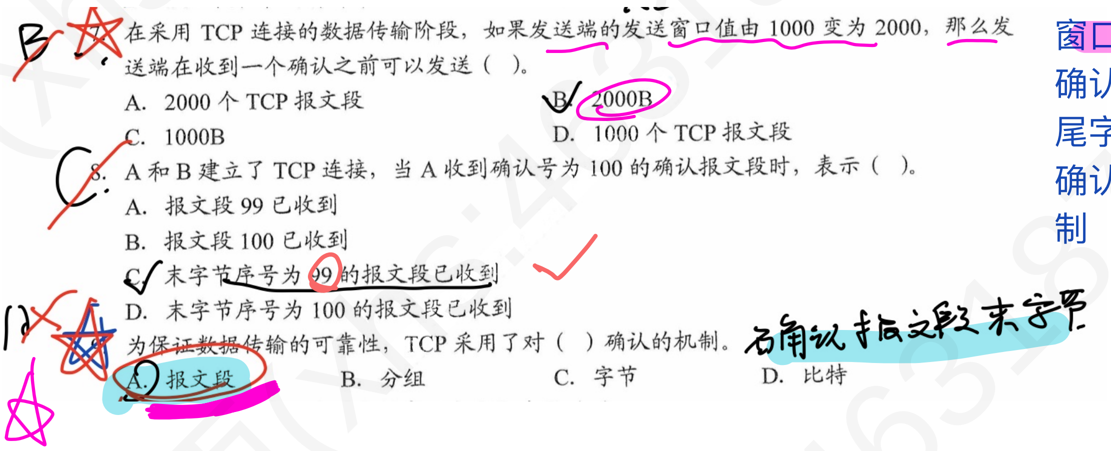
拥塞窗口是会增长的, ssthresh只是拥塞窗口的阈值; 网络拥塞时的cwnd只是那一时刻的cwnd, 这一点要判断清楚,

swnd = min{cwnd, rwnd}.

[15题]:拥塞窗口是**发送方**根据网络拥塞情况确定的窗口值(变化的)

swnd = min{cwnd, rwnd}

求拥塞窗口的题目可以按照左边这种数轴形式做



B 7. 在采用 TCP 连接的数据传输阶段，如果发送端的发送窗口值由 1000 变为 2000，那么发送端在收到一个确认之前可以发送（　）。

A. 2000 个 TCP 报文段　　B. 2000B

C. 1000B　　D. 1000 个 TCP 报文段

C 8. A 和 B 建立了 TCP 连接，当 A 收到确认号为 100 的确认报文段时，表示（　）。

A. 报文段 99 已收到

B. 报文段 100 已收到

C. 末字节序号为 99 的报文段已收到

D. 末字节序号为 100 的报文段已收到

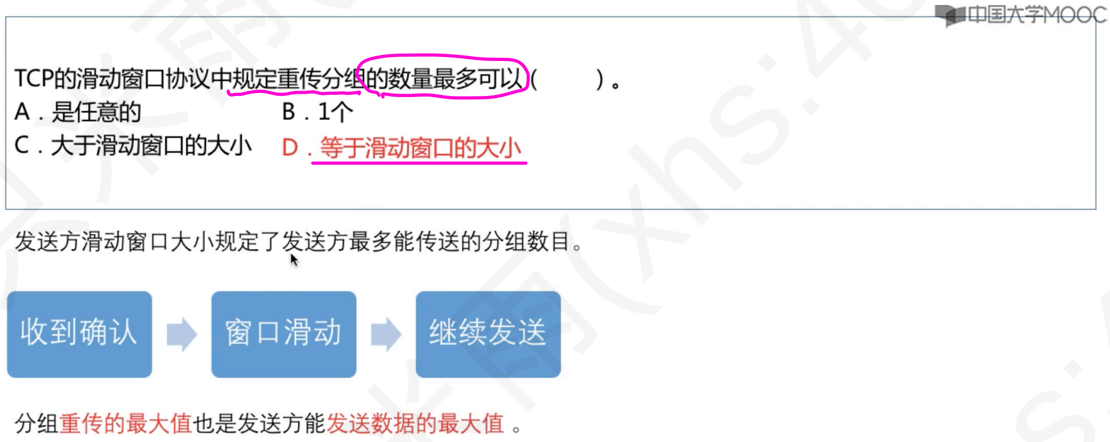
12. 为保证数据传输的可靠性，TCP 采用了对（　）确认的机制。确认报文段末字节

A. 报文段　　B. 分组　　C. 字节　　D. 比特

窗口大小的单位是: 字节;

确认序号是接收到的报文段末尾字节+1;

确认机制是对报文段的确认机制



TCP的滑动窗口协议中规定重传分组的数量最多可以（　　）。

A．是任意的　　B．1个

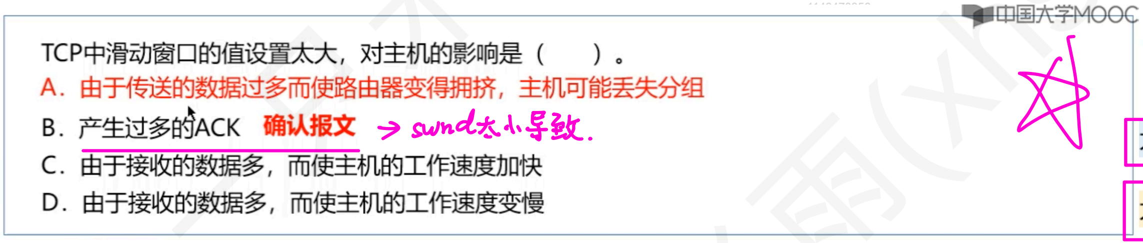
C．大于滑动窗口的大小　　D．等于滑动窗口的大小

发送方滑动窗口大小规定了发送方最多能传送的分组数目。

收到确认 ➡ 窗口滑动 ➡ 继续发送

分组重传的最大值也是发送方能发送数据的最大值。

13. 分组重传的最大值, 也就是发送方能发送数据的最大值



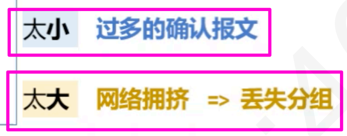
TCP中滑动窗口的值设置太大，对主机的影响是（　　）。

A．由于传送的数据过多而使路由器变得拥挤，主机可能丢失分组

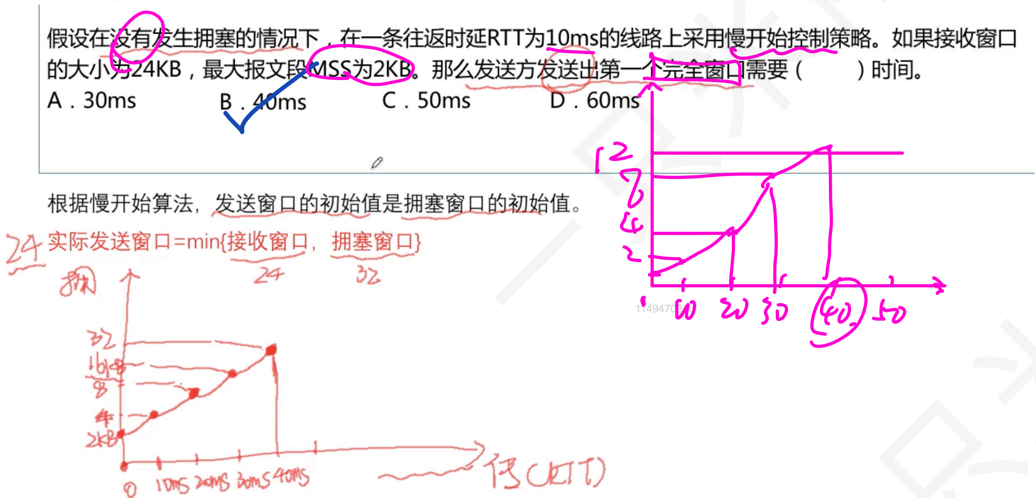
B．产生过多的ACK 确认报文 → swnd太小导致.

C．由于接收的数据多，而使主机的工作速度加快

D．由于接收的数据多，而使主机的工作速度变慢



太小	过多的确认报文
太大	网络拥挤 => 丢失分组



假设在没有发生拥塞的情况下，在一条往返时延RTT为10ms的线路上采用慢开始控制策略。如果接收窗口的大小为24KB，最大报文段MSS为2KB。那么发送方发送出第一个完全窗口需要（　　）时间。

A．30ms　　B．40ms　　C．50ms　　D．60ms

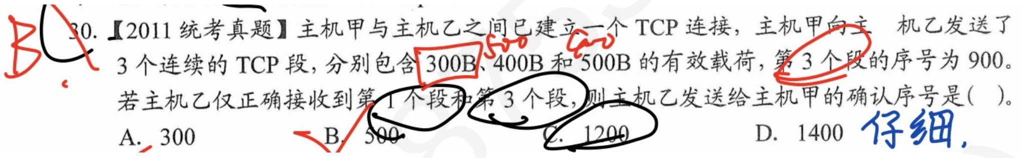
根据慢开始算法，发送窗口的初始值是拥塞窗口的初始值。

24 实际发送窗口=min{接收窗口，拥塞窗口}

24　32

25. 完全窗口就是在不发生拥塞的情况下, 发送窗口大小等于接收窗口大小;

只用考虑到40ms就算是发送完了第一个完全窗口了



B

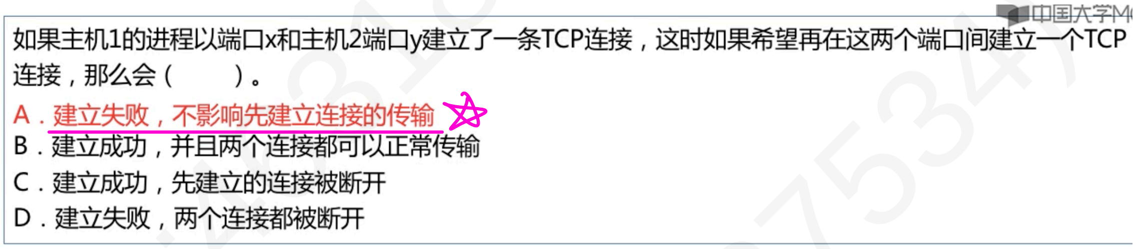
30.【2011 统考真题】主机甲与主机乙之间已建立一个 TCP 连接，主机甲向主机乙发送了 3 个连续的 TCP 段，分别包含 300B、400B 和 500B 的有效载荷，第 3 个段的序号为 900。若主机乙仅正确接收到第 1 个段和第 3 个段，则主机乙发送给主机甲的确认序号是（ ）。

A. 300　　B. 500　　C. 1200　　D. 1400

500 600

仔细，

仔细, 题中说的是第三个段的序号为900



如果主机1的进程以端口x和主机2端口y建立了一条TCP连接，这时如果希望再在这两个端口间建立一个TCP连接，那么会（　　）。

A．建立失败，不影响先建立连接的传输

B．建立成功，并且两个连接都可以正常传输

C．建立成功，先建立的连接被断开

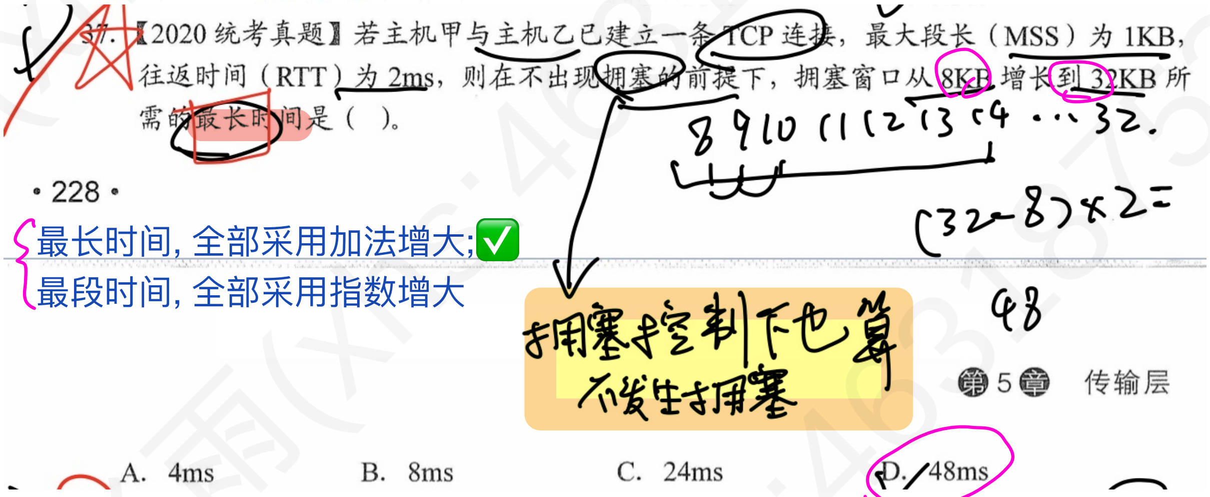
D．建立失败，两个连接都被断开

32. 重复建立的TCP连接会失败, 但不会影响已经建立的连接; 已经建立的连接只有数据传输完毕(不考虑意外),才会断开, 然后允许建立新的连接

套接字：（主机IP地址，端口号）

(1，x) - (2，y)

TCP　　有三次握手，有确认，有流量控制　　　　面向连接、可靠



37.【2020 统考真题】若主机甲与主机乙已建立一条 TCP 连接，最大段长（MSS）为 1KB，往返时间（RTT）为 2ms，则在不出现拥塞的前提下，拥塞窗口从 8KB 增长到 32KB 所需的最长时间是（ ）。

• 228 •

第 5 章　传输层

A. 4ms　　B. 8ms　　C. 24ms　　D. 48ms

8 9 10 11 12 13 14 … 32.

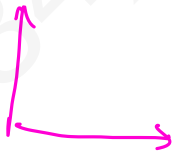
(32-8)×2=

48

拥塞控制下也算不发生拥塞

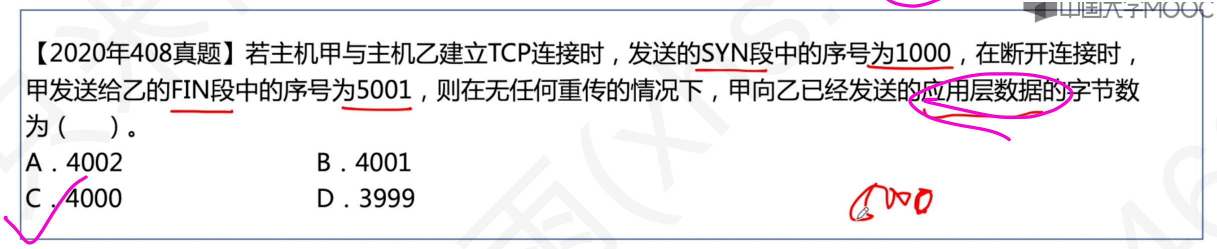
最长时间, 全部采用加法增大;✅

最段时间, 全部采用指数增大



32-8=24 ×2)

=

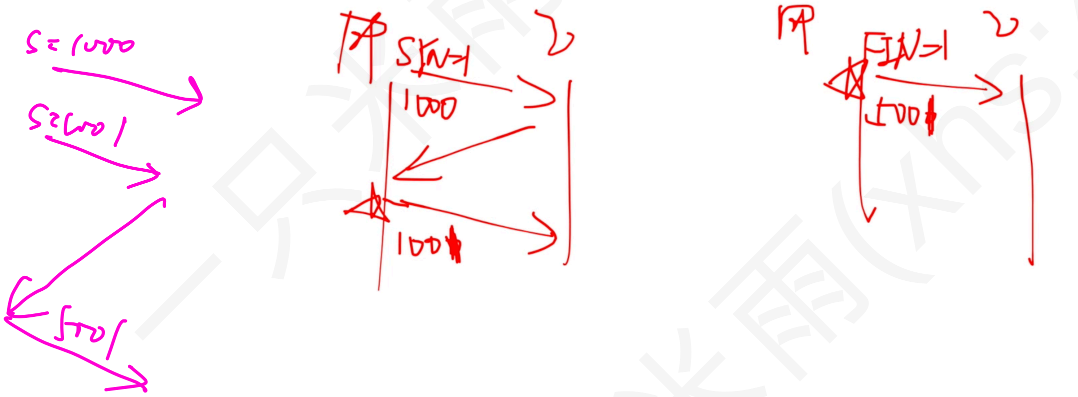


【2020年408真题】若主机甲与主机乙建立TCP连接时，发送的SYN段中的序号为1000，在断开连接时，甲发送给乙的FIN段中的序号为5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为（　　）。

A．4002　　B．4001

C．4000　　D．3999

5000



38.

连接三次握手的过程, 前两次都是没有发送数据的, 但是占用一个字节的序号, **应用层数据从第三次握手开始发送**, 即起始为1001;

FIN序号为5001,说明上一个报文段的最后一个字节为5000;

则共发送了:

5000-1001+1=4000

大题:

15. 以太网最小帧长为64B, 有效载荷范围: 46B-1500B (46 = 64-(6+6+2) - 4) [662IP4]

有效载荷小于46B, 需要在数据链路层中进行填充

16. [2016]第二问, 题目问的是发送端第8个段, 而不是第8轮传输; 这就需要用到自顶向下里面的知识点了, 慢启动阶段成功传送一个报文, 拥塞窗口就+1

第三问, H3建立连接时初始序号为100, 这是建立连接的报文段的序号, 建立连接的报文段不包含数据内容, 但是占用一个序号, 因此真正开始发送应用层数据的报文段的序号是从100+1=101开始的;

假设下图中的H3访问Web服务器S时，S为新建的TCP连接分配了20KB（K=1024）的接收缓存，最大段长MSS=1KB，平均往返时间RTT=200ms。H3建立连接时的初始序号为100，且持续以MSS大小的段向S发送数据，拥塞窗口初始阈值为32KB；S对收到的每个段进行确认，并通告新的接收窗口。假定TCP连接建立完成后，S端的TCP接收缓存仅有数据存入而无数据取出。【2016年计算机联考真题】

回答下列问题。 考点：TCP拥塞控制

12KB　9　9

2）H3收到的第8个确认段所通告的接收窗口是多少？此时H3的拥塞窗口变为多少？H3的发送窗口变为多少？

RTT　RTT　RTT

	0	+1	+1	+1	+1	+1	+1	+1	+1	
cwnd	1✓	2✓	3	4✓	5	6	7	8	9	
rwnd	20	19	18	17	16	15	14	13	12	
twnd	1	2	3	4	5	6	7	8	9	

$$t_{wnd} = \min[c_{wnd}, r_{wnd}]$$

回答下列问题。

3）当H3的发送窗口等于0时，下一个待发送的数据段序号是多少？H3从发送第1个数据段到发送窗口等于0时刻为止，平均数据传输速率是多少（忽略段的传输延时）？ 考点：TCP序号

$$t_{wnd} = \min[c_{wnd}, r_{wnd}]$$

数据传输速率=$\frac{\text{传输的数据长度}}{\text{时间}}$

$r_{wnd} = 0$

已经发送20KB的数据

$100+20KB=100+20*2^{10}=20580$

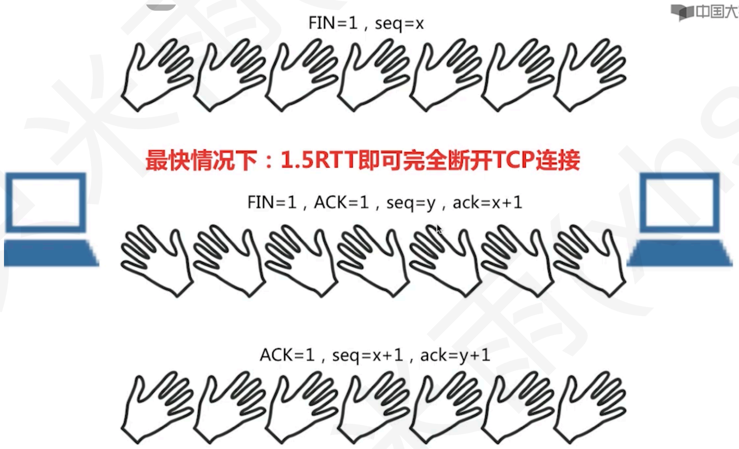
下一段20581

$$= \frac{20KB}{\text{时间}}$$

$$= \frac{20KB}{5RTT}$$

RTT　RTT　RTT　RTT　RTT

	0	+1	+2	+4	+8	
cwnd	1	2	4	8	16	
rwnd	20	19	17	13-8=	5	0
twnd	1	2	4	8	5	0



最快情况下：1.5RTT即可完全断开TCP连接

不用考虑client在哪里等待的2*MSL(两倍最长报文段寿命), 只要server受到最后对FIN的确认报文段, 就认为连接已经断开了

一、 msl、ttl及rtt的区别

1、 MSL 是Maximum Segment Lifetime英文的缩写，中文可以译为“报文最大生存时间”，他是任何报文在网络上存在的最长时间，超过这个时间报文将被丢弃。因为tcp报文（segment）是ip数据报（datagram）的数据部分，具体称谓请参见《数据在网络各层中的称呼》一文；

2、ip头中有一个TTL域，TTL是 time to live的缩写，中文可以译为“生存时间”，这个生存时间是由源主机设置初始值但不是存的具体时间，而是存储了一个ip数据报可以经过的最大路由数，每经 过一个处理他的路由器此值就减1，当此值为0则数据报将被丢弃，同时发送ICMP报文通知源主机。RFC 793中规定MSL为2分钟，实际应用中常用的是30秒，1分钟和2分钟等。

~~TTL~~RTT与MSL是有关系的但不是简单的相等的关系，MSL要大于等于~~TTL~~RTT。

3、 RTT是客户到服务器往返所花时间（round-trip time，简称RTT），TCP含有动态估算RTT的算法。TCP还持续估算一个给定连接的RTT，这是因为RTT受网络传输拥塞程序的变化而变化

4、2MSL即两倍的MSL，TCP的TIME_WAIT状态也称为2MSL等待状态，当TCP的一端发起主动关闭，在发出最后一个ACK包后，即第3次握 手完成后发送了第四次握手的ACK包后就进入了TIME_WAIT状态，必须在此状态上停留两倍的MSL时间，等待2MSL时间主要目的是怕最后一个 ACK包对方没收到，那么对方在超时后将重发第三次握手的FIN包，主动关闭端接到重发的FIN包后可以再发一个ACK应答包。在TIME_WAIT状态 时两端的端口不能使用，要等到2MSL时间结束才可继续使用。当连接处于2MSL等待阶段时任何迟到的报文段都将被丢弃。不过在实际应用中可以通过设置 SO_REUSEADDR选项达到不必等待2MSL时间结束再使用此端口。对于TCP中的各种控制字段，接下来进行具体说明。

只米雨(xhs:463187534)
1149470356